KUOTA · FINTECH DE CRÉDITO Y COBROS

Documentación técnica de ciberseguridad

Cómo se protegen las cuentas, el dinero y los datos: autenticación, sesiones, cobros firmados, webhooks, antifraude, trazabilidad inmutable y riesgos residuales.

Backend Express 5 · PostgreSQL | App Android (Capacitor + React) | Railway

Versión 2026-11-07 · Verificado contra el código fuente · Alcance: backend, app Android/web y despliegue
Clasificación: confidencial — uso interno y de inversionistas bajo NDA

Contenido

1. 1. Resumen ejecutivo
2. 2. Arquitectura y fronteras de confianza
3. 3. Gestión de secretos y configuración
4. 4. Identidad, PIN y recuperación
5. 5. Sesiones — `security/auth.ts`
6. 6. Dispositivos y enfriamiento
7. 7. Confirmación reforzada (step-up) y consentimiento
8. 8. Links de cobro y página de pago — `security/links.ts`, `routes/paymentLink.routes.ts`, `routes/payPage.routes.ts`
9. 9. Pagos entrantes: webhooks — `security/webhook.ts`, `routes/webhook.routes.ts`
10. 10. Antifraude y límites — `security/risk.ts`, `riskRules.ts`, `limits.ts`
11. 11. Integridad, trazabilidad y conciliación
12. 12. Aislamiento de datos y privilegio mínimo — `migrations/0003_roles.sql`
13. 13. Protección de la API y del cliente
14. 14. Riesgos residuales y plan de mejora (leer antes de operar con dinero real)
15. 15. Controles de verificación (pruebas automatizadas)
16. 16. Operación segura (checklist)

Todo lo descrito está verificado contra el código fuente; cada sección indica el archivo donde vive el control. La sección 14 lista **lo que NO está protegido todavía**: léela antes de salir a producción con dinero real.

---

1. Resumen ejecutivo

Kuota es una fintech para comercios pequeños: registro y verificación (KYC), línea de crédito en cuotas y cobros por link/QR. La seguridad se apoya en seis ideas:

1. **El PIN de 6 dígitos es el secreto del comercio**, guardado solo como hash scrypt con pimienta y protegido por bloqueo por intentos.
2. **Sesiones cortas (15 min) firmadas y ligadas al dispositivo**; las operaciones de dinero exigen sesión, `Idempotency-Key` y, si el riesgo lo pide, confirmación con PIN de un solo uso (*step-up*).
3. **Un link de cobro es un objeto firmado**: monto, titular y vencimiento no se pueden alterar (firma HMAC + triggers en la base).
4. **Los pagos solo se acreditan por webhook firmado** (HMAC, ventana de tiempo, idempotencia, conciliación).
5. **Antifraude por reglas editables en caliente**, límites por nivel de verificación y enfriamiento tras cambios sensibles.
6. **Trazabilidad inmutable**: bitácora encadenada con SHA-256 y tablas de dinero de solo agregado.

Modelo de amenazas cubierto: robo/adivinación de PIN, toma de cuenta, dispositivos nuevos no autorizados, manipulación de montos, replay/forja de webhooks, enumeración de links, doble cobro, fraude por ráfagas/estructuración, manipulación de la base y fuga de PII.

---

2. Arquitectura y fronteras de confianza



Zona	Contiene	Control
Núcleo (`public`)	comercios, usuarios, onboarding, crédito, ventas	rol `kuota_core_rw`
Dinero (`payments`)	links, pagos, libro, credenciales, sesiones, bitácora, riesgo	rol `kuota_payments_rw`
Interna	conciliación, riesgo, auditoría, flags	`Authorization: Bearer <KUOTA_ADMIN_TOKEN>`
Pública sin sesión	`/pay/:code` , `/payment-links/public/:code` , catálogos	rate limit + código firmado

Código de dos handles de base: `db` (núcleo) y `paymentsDb` (dinero) en `backend/src/db/client.ts` . Las transacciones se propagan con `AsyncLocalStorage` ; las secciones críticas se serializan con `pg_advisory_xact_lock` .

3. Gestión de secretos y configuración

Archivo: `backend/src/security/config.ts`.

Secreto	Uso
`KUOTA_SESSION_SECRET`	firma HMAC de los tokens de sesión
`KUOTA_PIN_PEPPER`	pimienta del hash de PIN (fuera de la base)
`KUOTA_LINK_SIGNING_KEY`	firma de links y MAC de la URL pública
`KUOTA_WEBHOOK_SECRET`	verificación HMAC de webhooks 360pay
`KUOTA_ADMIN_TOKEN`	rutas `/internal`
`DIDIT_WEBHOOK_SECRET`, `DIDIT_API_KEY`, `OPENPAY_PRIVATE_KEY`, `KUOTA_360PAY_API_KEY`	integraciones

- **Producción (`NODE_ENV=production`)**: cada secreto debe venir del entorno con ≥32 caracteres; si falta alguno **el proceso no arranca**.
- Desarrollo: si faltan, se generan una vez en `.dev-secrets.json` (modo 0600, ignorado por git).
- `.env`, `.env.*` y `.dev-secrets.json` están en `.dockerignore` / `.gitignore`: no entran a la imagen.
- Interruptores relevantes: `KUOTA_SANDBOX` (ver §14), `KUOTA_TRUST_PROXY`, `KUOTA_CORS_ORIGINS`, `KUOTA_AUTO_MIGRATE`.

Contenedor (`backend/Dockerfile`): build multi-etapa, imagen `node:24-alpine`, `npm ci --omit=dev`, ejecuta como `USER node` (no root), sin `src/test/.env` en la imagen final.

4. Identidad, PIN y recuperación

4.1 Almacenamiento del PIN — `security/crypto.ts`

- PIN de **exactamente 6 dígitos**; se rechazan 14 combinaciones triviales (`123456`, `000000`, repetidos…).
- Hash: `scrypt(HMAC-SHA256(pimienta, PIN), sal aleatoria de 16 B, N=16384, r=8, p=1, 32 B)`. Formato `scrypt$N$r$p$sal$hash`.
- Comparación con `timingSafeEqual`. Un PIN solo tiene 10^6 combinaciones: la **pimienta fuera de la base** y el **bloqueo** son lo que impide probarlas todas con una copia de la base.
- El PIN nunca se registra: la bitácora redacta las claves `pin`, `current_pin`, `new_pin`, `access_token`, `step_up_token`, `token`, `secret`.

4.2 Bloqueo por intentos — `auth.ts › checkPin`

- Producción: **5 intentos fallidos → bloqueo 15 min** (HTTP 423 `PIN_LOCKED`); el contador sube con un `UPDATE … RETURNING` atómico (intentos simultáneos no se pierden). Un acierto resetea el contador.
- El bloqueo emite alerta al comercio (`PIN_LOCKED`) y asiento en bitácora.
- Respuesta 401 `PIN_INVALID` informa intentos restantes.

4.3 Login — `POST /auth/sessions`

1. Resuelve el comercio (`resolveClaimedMerchant`): cabecera `X-Merchant-Id` si existe; si es obsoleta o falta, por teléfono (`users.phone` → `merchant_profiles`), con respaldo por `merchant_profiles.phone` . En producción jamás se cae a una cuenta demo.
2. Rechaza dispositivos revocados (403 `DEVICE_REVOKED`).
3. Verifica el PIN (§4.2).
4. Vincula el dispositivo (§6) y emite sesión (§5).

4.4 Recuperación de PIN — `POST /auth/pin/recover`

Para quien olvidó el PIN. Prueba de identidad: **teléfono + DNI del registro**. Controles añadidos tras la auditoría:

- **Misma respuesta (403 `DNI_MISMATCH`) si la cuenta no existe o el DNI no coincide** → no se pueden enumerar teléfonos registrados.
- **Límite por cuenta**: 5 DNI fallidos / 15 min → 429 `RECOVERY_LOCKED` (el DNI es semipúblico; sin esto se adivinaría cambiando de IP).
- Valida el dispositivo **antes** de tocar nada (un dispositivo revocado no puede usar la recuperación).
- Al tener éxito: cambia el PIN, **revoca todas las sesiones**, aplica **enfriamiento de 24 h** (límites al 25 %), emite alerta `PIN_CHANGED` y asienta `PIN_RESET` en bitácora.
- Además, `POST /onboarding/sessions/:id/security-pin` ya no permite pisar el PIN de una cuenta que ya tiene credencial (409 `PIN_ALREADY_SET`).

4.5 Cambio de PIN — `POST /auth/pin/change`

Exige sesión + PIN actual + PIN nuevo distinto y no débil; cierra las demás sesiones, aplica enfriamiento 24 h y alerta.

---

5. Sesiones — `security/auth.ts`

Token `ks1.<payload base64url>.<HMAC-SHA256>` ; payload `{sid, mid, did, exp}` .

Validación en cada petición (`requireMerchant`), **todas** deben cumplirse:

1. firma HMAC válida (comparación en tiempo constante) y no vencido (**TTL 15 min**, `KUOTA_SESSION_TTL_SEC`);
2. `X-Device-Id` igual al `did` firmado en el token (token robado ≠ sesión utilizable en otro equipo);
3. fila en `merchant_sessions` no revocada ni expirada;
4. dispositivo no revocado.

Revocación: `DELETE /auth/sessions/current` , cambio/recuperación de PIN, desvincular dispositivo y `POST /internal/sessions/revoke` . El `device_id` debe cumplir `^[A-Za-z0-9_-]{16,64}$` .

---

6. Dispositivos y enfriamiento

- Cada ingreso vincula `merchant_devices(merchant_id, device_id, label, first_seen, last_seen, cooldown_until, revoked_at)` .
- **Dispositivo nuevo cuando ya hay otro activo** → `cooldown_until = +24 h` , alerta `NEW_DEVICE` y límites reducidos al **25 %** (`cooldownFactor`). Serializado con advisory lock por comercio (dos primeros ingresos simultáneos no esquivan el enfriamiento).

- `GET/DELETE /merchants/me/payments/devices`: listar y desvincular otro dispositivo (exige step-up con PIN; cierra sus sesiones; no se puede desvincular el actual).
- El mismo enfriamiento aplica tras cambiar/recuperar el PIN (`merchant_credentials.cooldown_until`).

---

7. Confirmación reforzada (step-up) y consentimiento

Acciones: `CREATE_LINK`, `RAISE_LIMITS`, `REVOKE_DEVICE` (`auth.ts`).

Flujo: la ruta responde 403 `STEP_UP_REQUIRED` con resumen legible → el cliente pide el PIN → `POST /auth/step-up` valida el PIN, guarda un **registro de consentimiento** (`consent_records`: resumen, parámetros canónicos, IP, dispositivo, sesión) y emite un token aleatorio de 192 bits.

El permiso:

- se guarda **solo como hash SHA-256**;
- es de **un solo uso** (`UPDATE … WHERE used_at IS NULL`, atómico);
- vence en **120 s**;
- está **atado a la sesión, la acción y el hash de los parámetros exactos** (un permiso para S/ 50 no sirve para S/ 500).

---

8. Links de cobro y página de pago — `security/links.ts`, `routes/paymentLink.routes.ts`, `routes/payPage.routes.ts`

Control	Detalle
Token	128 bits aleatorios (`base64url`, 22 car.)
Código público	`<token>.<MAC>` con MAC HMAC de 12 car.: un código inventado se rechaza **antes de consultar la base** (anti-enumeración)
Firma de fila	HMAC de `id, comercio, token, monto, comisión, neto, moneda, vencimiento`; si se altera → `TAMPER_DETECTED`, el link deja de ser pagable y se alerta al comercio
Inmutabilidad	trigger `payment_links_guard`: monto/titular/vencimiento inmutables, la firma no se reemplaza, un link pagado no cambia de estado
Vigencia	48 h; estados `DRAFT → ACTIVE → PAID/EXPIRED/CANCELLED/ON_HOLD`
Idempotencia	creación y emisión exigen `Idempotency-Key` (16–80 car.): misma clave+cuerpo devuelve la respuesta original; otro cuerpo → 409; reserva previa evita doble ejecución concurrente
Montos	enteros en céntimos, mínimo y máximos por nivel; se rechazan decimales/negativos
Propiedad	todas las rutas de dueño filtran por `merchant_id` de la sesión (protección BOLA)

Página `/pay/:code`: HTML servido por el backend sin sesión; CSP propia (`default-src 'self'; script-src 'unsafe-inline'; connect-src 'self'; frame-ancestors 'none'`); datos dinámicos se escapan con `escHtml`; los botones usan listeners (sin atributos inline) y el origen de la API es relativo (no se inyecta la cabecera `Host`).

---

9. Pagos entrantes: webhooks — `security/webhook.ts`, `routes/webhook.routes.ts`

360pay (`POST /webhooks/360pay`) — un pago solo se acredita aquí:

1. cuerpo crudo conservado (`rawBody`) para verificar la firma;
2. `X-360Pay-Timestamp` dentro de **±300 s** (anti-replay);
3. firma **HMAC-SHA256** sobre `"<timestamp>.<cuerpo>"` con `KUOTA_WEBHOOK_SECRET` , comparación constante;
4. `X-360Pay-Hook-Id` igual al configurado;
5. `X-360Pay-Event-Id` obligatorio; la misma id con **otro contenido** → 409 `EVENT_ID_REUSED` ; reentrega idéntica → respuesta guardada (`duplicate: true`), sin reprocesar;
6. todo el evento en **una transacción** con lock por event-id;
7. por cupón: link existente, **firma del link válida**, `gateway_ref` no repetido, link `ACTIVE` , **monto exacto** (si no, rechazo + alerta `AMOUNT_MISMATCH`), un solo pago por link (segundo cobro → excepción de conciliación);
8. crédito atómico: `payments` + libro de doble partida (`ledger_entries`) + puntos de score;
9. toda decisión no exitosa queda en bitácora con código y motivo.

Didit (KYC) — `verifyDiditWebhook` : `X-Timestamp` ±300 s y HMAC-SHA256 (hex) con `DIDIT_WEBHOOK_SECRET` (acepta firma V2 canónica, firma sobre cuerpo crudo y variantes `*-previous` para rotar secretos).

Openpay (enganche) — firma HMAC-SHA1 base64 con clave privada, comparación en tiempo constante; **sin clave configurada en producción se rechaza todo** (falla cerrado).

Interruptor de incidente: `feature_flags` (`WEBHOOK_PROCESSING` , `LINK_CREATE` , `PUBLIC_PAY` …) se apaga sin redeploy vía `PUT /internal/features/:name` .

---

10. Antifraude y límites — `security/risk.ts` , `riskRules.ts` , `limits.ts`

10.1 Límites por nivel (rige el más restrictivo)

Nivel	Por cobro	Por día	Cobros/día	Step-up desde
Básico	S/ 1 000	S/ 3 000	50	S/ 500
Ampliado (KYC aprobado)	S/ 5 000	S/ 20 000	200	S/ 2 000

Efectivo = mín(nivel, límite propio, tope 360pay, ×0,25 si hay enfriamiento). **Bajar** un límite rige de inmediato; **subirlo** exige PIN y entra en vigor tras **24 h** (cancelable sin PIN: `DELETE /limits/pending`).

10.2 Reglas de riesgo (tabla `risk_rules` , editables, decisión APPROVE < STEP_UP < REVIEW < BLOCK)

`list_black` (BLOCK), `list_gray` (REVIEW), `high_amount` , `amount_vs_history` (>5× promedio), `link_velocity_review/block` (10/25 links en 10 min), `new_merchant_volume` , `structuring` (cobros fraccionados bajo el 90 % del límite), `unpaid_spike` , `night_high_amount` (00–05 h Lima), `device_cooldown` , `shared_device` , `card_testing` (5 fallos/10 min), `payer_velocity` . Listas negra/gris/blanca por comercio, dispositivo e IP; la blanca exime de retenciones blandas, **nunca de un bloqueo**. Cada decisión se persiste (`risk_decisions` , con latencia y motivos). Los links retenidos pasan a `ON_HOLD` con revisión manual (`/internal/risk/reviews`).

---

11. Integridad, trazabilidad y conciliación

- **Bitácora encadenada** (`audit.ts`): cada fila guarda `prev_hash` y `hash = SHA-256(ts, actor, acción, objetivo, resultado, IP, dispositivo, detalles, prev_hash)` ; escritura serializada con advisory lock. `GET /internal/audit/verify` recorre la cadena y detecta alteración, borrado o reordenamiento.

- **Solo agregado en la base**: triggers `BEFORE UPDATE OR DELETE` y `BEFORE TRUNCATE` sobre `payments`, `ledger_entries`, `audit_log`, `payment_events`, `consent_records`, `risk_decisions`, `webhook_events`, `payment_attempts`, `merchant_alerts`, `reconciliation_runs`; además el rol `kuota_payments_rw` **no tiene** `UPDATE/DELETE` sobre ellas (doble barrera).
- **Libro de doble partida** cuadrado por pago.
- **Conciliación** (`reconciliation.ts`): cruza link ↔ pago ↔ libro ↔ webhooks ↔ reporte de 360pay y marca excepciones (capturado no abonado, diferencia de monto, firma rota, etc.).
- **Alertas al comercio** (`events.ts`): PIN bloqueado/cambiado, dispositivo nuevo, límites, links, cobros alterados.

---

12. Aislamiento de datos y privilegio mínimo — `migrations/0003_roles.sql`

- `kuota_core_rw`: CRUD en `public`; **sin** acceso al esquema `payments` (probado: `permission denied`).
- `kuota_payments_rw`: CRUD solo donde necesita; del núcleo únicamente lee `merchant_profiles`, `users`, `onboarding_sessions`, inserta `sales` / `score_events` y actualiza la columna `score`.
- `schema_migrations` no es escribible por el núcleo. Las migraciones llevan checksum: editar una ya aplicada falla.
- En producción hay un usuario por zona (`DATABASE_URL`, `PAYMENTS_DATABASE_URL`). **En el despliegue actual de Railway solo existe `DATABASE_URL`: ambos handles usan el mismo usuario y el aislamiento por roles no está activo** (§14).

---

13. Protección de la API y del cliente

13.1 Backend (`app.ts`)

- Cabeceras en toda respuesta: `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`, `Cache-Control: no-store`; en producción también `Strict-Transport-Security` (1 año, subdominios). `x-powered-by` deshabilitado.
- **CORS**: sin `Origin` (app nativa, webhooks) permitido; con `Origin`, lista explícita (`http://localhost`, `capacitor://localhost` en producción).
- Cuerpo JSON máx. **200 kB**; JSON inválido → 400 sin detalles.
- **Rate limit** por IP (en memoria): global 600/min; `/auth/*` sensibles 10/min; link público 30/min; simuladores 30/min.
- Errores 5xx en producción: mensaje genérico, sin pila ni detalles internos; `console.error` solo con código y mensaje.
- Manejo de errores centralizado; Express 5 captura rechazos de promesas asíncronas.
- Acceso a datos con **consultas parametrizadas** (`?` / `@nombre` → `$n`); no hay SQL concatenado con entrada del usuario (la búsqueda de ventas escapa `%`, `_`, `!` y elimina NUL).
- `KUOTA_TRUST_PROXY=1` en Railway: se confía en el primer salto del proxy para `req.ip`.

13.2 Identidad en rutas del panel (`security/merchantContext.ts`)

Las rutas `/merchants/me/*` y `/credit/*` identifican al comercio con `X-Merchant-Id`; **en producción un id ausente o inexistente devuelve 404** `MERCHANT_NOT_FOUND` (nunca otra cuenta). Pagar cuota, liquidar compra y crear compra verifican que el recurso pertenezca a ese comercio, son atómicos e idempotentes, y la creación de compra se serializa por comercio (sin carreras sobre el límite de crédito).

13.3 Proveedores

`POST /businesses/confirm` entrega un **token secreto** (se guarda solo su SHA-256, columna `access_token_hash`, migración 0009). `PUT /providers/activity|settlement-account` y `POST /providers/activate` exigen `X-Provider-Id` + `X-Provider-Token`. Registrar un RUC ya existente responde 409 y no modifica nada.

13.4 Onboarding y PII

- Las sesiones (`ses_…`) no son secretas: `GET /onboarding/sessions/:id` devuelve solo campos no sensibles; una sesión `COMPLETED` no admite reescritura de identidad/negocio/productos; el celular de la cuenta sale del paso de identidad, no del cuerpo de `/complete`; el choque de unicidad devuelve 409.
- Estado de KYC (`/merchant-kyc/:id` , `/kyc/requests/:id`): DNI y teléfono **enmascarados**; id desconocido → 404 en producción.
- KYC real con Didit (documento + prueba de vida + coincidencia de DNI); en producción el modo DEMO se rechaza.

13.5 Cliente (APK / web)

- Almacena en `localStorage` : `kuota.merchant_id` , `kuota.provider_id` / `provider_token` , id de dispositivo aleatorio y banderas de UI. **No guarda PIN** ni token de sesión en disco (el access token vive en memoria y dura 15 min).
- El PIN se pide con teclado propio, se envía por HTTPS solo a `/auth/*` y se borra del estado tras usarlo.
- Cierre de sesión: revoca la sesión en servidor y limpia comercio, proveedor y token.
- Mensajes de error en español sin filtrar detalles técnicos.

---

14. Riesgos residuales y plan de mejora (leer antes de operar con dinero real)

#	Riesgo	Severidad	Estado	Mitigación recomendada
R1	`KUOTA_SANDBOX=1` **activo en producción**: cualquiera con el código de un link puede marcarlo pagado (`/sandbox/360pay/pay`) y confirmar enganches (`/sandbox/openpay/pay`). Los pagos son ficticios	**Crítica si hay dinero real**	Aceptado solo para pruebas	Configurar 360pay y Openpay reales y **quitar la variable**
R2	**Identidad por cabecera** `X-Merchant-Id` en `/merchants/me` y `/credit` (el id es un identificador, no un secreto; aparece en respuestas)	**Alta**	Mitigado parcialmente (404 sin id)	Derivar el comercio del `access_token` firmado en todas las rutas; no aceptar el id del cliente
R3	Recuperación de PIN basada en teléfono + DNI (ambos semipúblicos)	Media-alta	Mitigado (límite por cuenta, alerta, enfriamiento)	Añadir OTP por SMS/WhatsApp o prueba de vida (Didit) y enfriar más
R4	KYC de comercio/proveedor aprobado en modo demo si Didit no está configurado	Alta	**Parcial**: `/providers/activate` ya no auto-aprueba (el estado solo cambia por decisión de verificación)	Configurar Didit

#	Riesgo	Severidad	Estado	Mitigación recomendada
R5	Roles de base no aplicados (un solo usuario `DATABASE_URL`)	Media	Abierto	Crear usuarios `kuota_core_rw` / `kuota_payments_rw` y definir `PAYMENTS_DATABASE_URL`
R6	Rate limit y contadores de recuperación **en memoria** (se reinician con cada deploy y no se comparten entre réplicas)	Media	Abierto	Redis o límite en el API Gateway/WAF
R7	Bloqueo de PIN denegable por terceros (5 PIN erróneos con solo el teléfono bloquean 15 min)	Media	Diseño	CAPTCHA/Play Integrity, bloqueo por dispositivo+cuenta
R8	`androidScheme: http` y `cleartext` en Capacitor	Media	**Parcial**: `allowBackup=false` aplicado (sin copia de datos de la app)	`androidScheme: https` + `cleartext: false` (requiere migrar el almacenamiento local y ajustar CORS), *certificate pinning*
R9	Sin firma/ofuscación de release ni detección de root/emulador; el APK actual es `debug`	Media	Abierto	Build release firmado, R8/ProGuard, Play Integrity API
R10	Scoring y niveles manipulables o incoherentes (señales, progresión, mora inexistente)	Media	Parcial (señales con lista cerrada y sin duplicados)	Reglas de nivel y vencimientos basadas en eventos verificados
R11	El enganche y las cuotas de crédito **no cobran dinero real** (solo cambian estado)	Alta	Abierto	Cobro real por cuota con confirmación por webhook antes de marcar `PAID`
R12	Sin cifrado a nivel de columna de PII (DNI, teléfono, CCI) ni política de retención	Media	Abierto	Cifrado de columnas con KMS, minimización y retención/ARCO (Ley 29733)
R13	Sin monitoreo externo (Sentry, métricas) ni backups probados	Media	**Parcial**: el servidor verifica la cadena de la bitácora al arrancar y cada hora y registra `[ALERTA-SEGURIDAD]` si se rompe	Observabilidad, alertas sobre ese log, backups con prueba de restauración, rotación de secretos
R14	Compras a crédito: la lista de recomendados es fija	Media	**Parcial**: la compra exige proveedor existente y no rechazado (o uno de los afiliados fijos)	Reemplazar la lista fija por la tabla real de proveedores afiliados

15. Controles de verificación (pruebas automatizadas)

`backend/test/*.test.ts` (node:test, 63 pruebas): migraciones y roles, PIN hash/bloqueo/sesión ligada al dispositivo, token alterado, links (monto, BOLA, inmutabilidad, vencimiento), step-up de un solo uso, límites y enfriamiento, webhook firmado (ventana, firma, monto, repetición), antifraude, bitácora (cadena rota al alterar), KYC con Didit simulado, 360pay simulado y regresiones de la ronda de QA (`hardening.test.ts`): dueño en pagos de cuotas, no pisar PIN, sesión de onboarding sin fuga, recuperación de PIN, validación de compras, carreras, búsqueda y señales.

Auditoría de QA: 8 agentes especializados atacaron la API contra una réplica local de producción (68 hallazgos, de los cuales se corrigieron los críticos y altos de backend; ver §14 para lo pendiente).

16. Operación segura (checklist)

1. Variables obligatorias definidas (§3) y rotadas al menos cada 90 días; `KUOTA_ADMIN_TOKEN` distinto por entorno.
2. `KUOTA_SANDBOX` **sin definir** en producción real.
3. Ejecutar `GET /internal/audit/verify` periódicamente y alertar si `ok=false`.
4. Revisar `risk/reviews` y `reconciliation/runs` a diario; resolver excepciones de "capturado no abonado".
5. Ante incidente: apagar `PUBLIC_PAY`, `LINK_CREATE` o `WEBHOOK_PROCESSING` con feature flags y revocar sesiones con `/internal/sessions/revoke`.
6. Deploy: migraciones con usuario dueño del esquema (`npm run migrate`) y `KUOTA_AUTO_MIGRATE=0` cuando existan roles separados.
7. Mantener dependencias al día (`npm audit`) y revisar las integraciones externas (360pay, Didit, Openpay) tras cada cambio de contrato.